JAVA CHAPTER 9 — GENERATED CODE: RECORDS, LOMBOK, PROXIES & ANNOTATION PROCESSORS

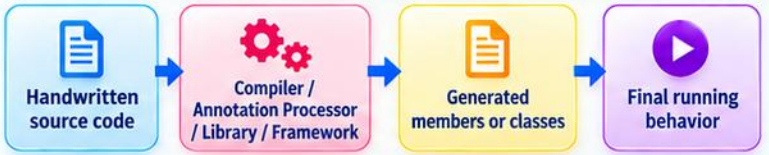
Understand code you did not write • Know what the compiler or framework creates

JAVA 17+

1 WHY IT MATTERS

- Modern Java projects often rely on generated code.
- It reduces boilerplate and speeds up development.
- Generated code can hide important behavior, making it harder to understand, debug and maintain applications.

2 MENTAL MODEL



Your classes, annotations, configuration

Processes code, analyzes annotations, applies rules

Additional code (e.g. methods, classes, proxies, metadata)

Executed by the JVM at compile time or runtime

3 COMMON SOURCES OF GENERATED CODE

Source	What it generates
Records	Constructor, accessors, equals, hashCode, toString and other methods.
Lombok	Getters, setters, constructors, builders and more via annotations.
Annotation Processors	New source files or additional members based on annotations.
Dynamic Proxies	Proxy classes that implement interfaces at runtime (e.g. java.lang.reflect.Proxy).
ORM / Framework Proxies	Subclass or proxy classes for lazy loading, transactions, security, dependency injection.
IDE-Generated Methods	Methods like getters/setters, equals/hashCode, toString, constructors and more.

4 RECORDS

Records automatically generate common members.

Java

```
public record Person(String name, int age) { }
```

Generated members include:

- Canonical constructor (Person(String name, int age))
- Accessors (name(), age())
- equals(Object)
- hashCode()
- toString() (e.g. Person[name=Ana, age=30])

Example toString() output:

```
Person[name=Ana, age=30]
```

5 LOMBOK

Lombok generates boilerplate code using annotations.

Java

```
import lombok.*;
@Data
@Builder
public class User {
    private String name;
    private int age;
}
```

Common Lombok annotations

@Getter	Generates getter methods.
@Setter	Generates setter methods.
@Builder	Builder API for object creation.
@Data	Combines getters, setters, toString, equals and hashCode.
@RequiredArgsConstructor	Constructor for final fields.

- Works via annotation processing and code generation.
- Methods are not visible in the source file but are generated.
- Requires the Lombok dependency and IDE support.

6 ANNOTATION PROCESSORS

Compile-time tools that generate source code.

Java

```
@Mapper
public interface UserMapper {
    UserDto toDto(User user);
}
```

Common use cases

Mapping code	e.g. object mappers
Serialization	e.g. JSON, XML, protocol buffers
DSL or builders	e.g. query builders
Client generators	e.g. REST or gRPC clients
Other tools	e.g. configuration, metadata

- Runs during compilation and generates `.java` source files.
- Generated sources are usually in `target/generated-sources` or `build/generated`.
- Examples: mapping libraries, serialization tools, configuration and more.

7 RUNTIME PROXIES

Frameworks may create proxy classes at runtime.

- **Transactions** — add transaction boundaries.
- **Lazy loading** — load data on demand.
- **Security** — enforce access control.
- **Logging** — add logging or monitoring.
- **Dependency injection** — manage bean lifecycles.

Proxies can implement interfaces or extend classes, and often add cross-cutting behavior at runtime.

8 HOW TO INSPECT GENERATED CODE

IDE navigation	Go to declaration, open generated sources, use IDE features to view generated members.
Decompiled class view	Use "Show bytecode" or a decompiler to view compiled classes.
Generated sources folder	Check target/generated-sources or build/generated-sources folders.
Stack traces	Generated classes often appear in stack traces. Useful for proxies or generated implementations.
Framework proxy names	Look for common naming patterns (e.g. $Proxy, $$EnhancerBySpringCGLIB$$).

9 COMMON MISTAKES

- Assuming behavior exists only in handwritten source code.
- Forgetting that equals, hashCode or toString are generated (e.g. records or Lombok).
- Not realizing proxies change the actual class type and can intercept methods.
- Debugging issues without checking generated code.
- Overusing boilerplate-hiding annotations and losing control of application behavior.

10 QUICK RULES / AT A GLANCE

- Records are great for immutable data carriers.
- Lombok is useful but requires team discipline.
- Inspect generated code when debugging or seeing unexpected behavior.
- Know the difference between compile-time and runtime generation.
- Generated code can improve productivity, but always understand what it creates.

11 JAVA EXAMPLE

Record vs Lombok-like example

Java

```
// Record (generated members)
public record Person(String name, int age) { }
// Lombok-style (conceptual)
@Data
class User {
    private String name;
    private int age;
}
```

Example output

```
System.out.println(new Person("Ana", 30));
// Output:
Person(name=Ana, age=30]
```